Plan of the chapter



1. Introduction
2. SQL data types
3. Common SQL Commands
4. SQL operators
5. Database and relational database design
6. Database normalization

What is the relational model?

- The relational model is built around three main concepts which are:
  - **Data,**
  - **Relationships,**
  - **Constraints.**
- It describes a database as “a collection of inter-related relations (or tables)”.
- It is still a dominant model used for data storage and retrieval.
- It is a way of organizing or storing data in a database.
- SQL is the language that’s used to retrieve data from a relational database.

Fundamental concepts of the relational model

- **Relation:** It represents a file that stores data. It's also known as a **table**.
- Within a table there are **rows** and **columns**.
- Each row represents a group of related data values. A row, or record, is also known as a tuple. It consists of a set of attributes.
- Columns in a table are also known as fields or attributes. These columns define or describe a row.

	Column 1	Column 2	Column 3
Row 1			
Row 1			
Row 1			

Fundamental concepts of the relational model

❖ **Column:** A table stores pieces of data or facts as columns (attribute).

- When determining the columns for your table, think about the pieces of data that need to be stored within that table.
- Each column is a generic representation of the piece of data that needs to be stored.
- Each table cell that becomes a part of a row will have a specific instance of a piece of data.

ID	First Name	Last Name
1	John	Smith
2	Mish	Azerrad
3	Peter	Klien

Fundamental concepts of the relational model

- **Domain:** The domain is a set of acceptable values that a column is allowed to contain.
  - The domain depends on the data type of the column whether it is numeric, or text based.
  - The domain of ID has a set of acceptable and possible values that are numeric such as 1, 2 and 3.
  - The domain of First Name has a set of acceptable and possible values that are text based, which is people's first names.
  - In the ID column, it's not possible to store values such as "Jane" or "0001". Similarly, the First Name column can't accept any numeric pieces of data.

Fundamental concepts of the relational model

❖ **Record or tuple:** It is a row within a table.

- If a table has columns for ID, First Name and Last Name, then one record or tuple would have one person's ID, first name and last name.
- Another record would have another person's full personal information.

❖ **Key:** Each row or tuple has one or more attributes, known as a relation key, that can uniquely identify a specific row.

➢ This is also known as the **primary key**.

Fundamental concepts of the relational model

❖ **Degree:** It is the number of columns or attributes within a relation.

- A student table that stores the student's name, address, phone number and email address would have a degree of **4**.

❖ **Cardinality:** It refers to how many records there are within a particular table in a database.

- If you have 100 students in your student table, with all their information organized into individual rows, then that table has a cardinality of 100.

What are constraints?

- In the relational model, every relation needs to meet three conditions.
- These three conditions must be met for a relation to be valid.
- They are called relational integrity constraints and they are:
  - **Key constraints**
  - **Domain constraints**
  - **Referential integrity constraints**

What are constraints?

- **Key constraints:** It revolves around the key attribute(s).
- In the relational model, a key attribute is an identifier that can be used to refer to a record.
- A key attribute must also be unique for each record.
  - For example, you can use the Student ID in the student table as the key.
    - There can't be two students with the same Student ID.
    - It would be invalid and cause an issue when it comes to accessing or retrieving the data.
- A key attribute cannot have NULL values.

What are constraints?

- **Domain constraints:** There are all about the requirement of inserting values that have a valid data type.
- There are a variety of data types that can be included within a table, namely numeric, text and data.
- If an attempt is made to store an incorrect data typed value to an attribute, it's declared a violation of domain constraints.
- If an attribute requires a numeric value to be entered, and the value you are attempting to enter uses letters instead of numbers, then it would be invalid.

What are constraints?

- **Referential integrity constraints:** A database has multiple tables that refer to one another. Referential integrity constraints are based on the concept of **foreign keys**.
- A foreign key is a key attribute present in a table, which is also a primary key of another table to which it needs to be linked.
  - The foreign key references the other table to which it's related.
  - For example, the order ID is present in the Order_Item table as a foreign key, which is also the primary key of the order table.
  - So, the order table and the Order_Item table are related to each other because the Order_Item table references the order table via the order ID attribute.

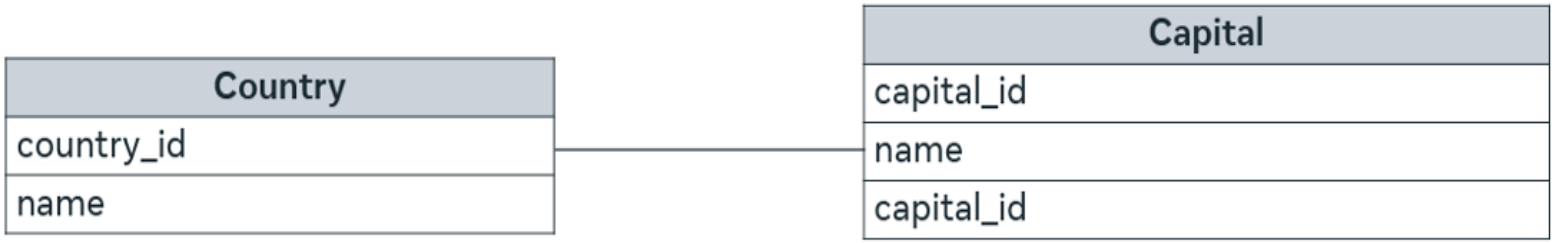
Types of relationships

- In the relational model, there are three types of relationships that can exist between tables:
  - **One-to-one**
  - **One-to-many**
  - **Many-to-many**

Types of relationships

❖ **One-to-one:** Let's take the example of two tables: Table A and Table B.

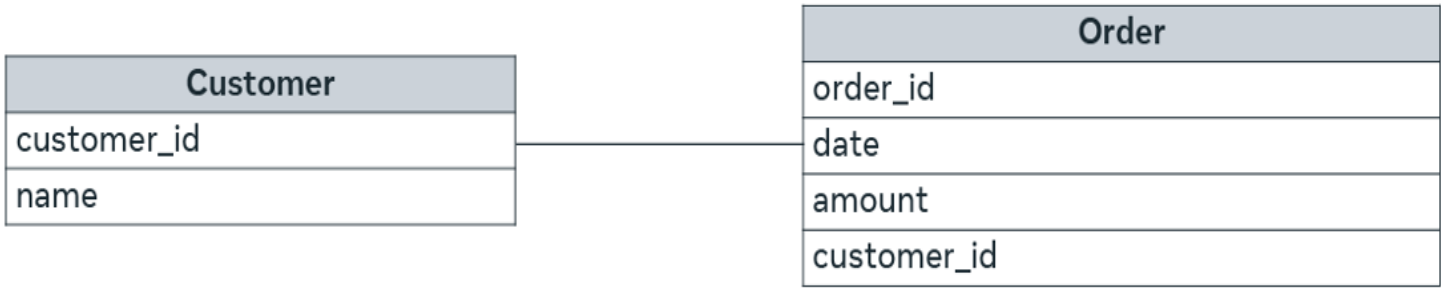
- A one-to-one (1:1) relationship means that each record in Table A relates to one, and only one, record in Table B. Likewise, each record in Table B relates to one, and only one, record in Table A.



- *Every country has one, and only one, capital.*
- *Every capital belongs to one and only one country.*

Types of relationships

- **One-to-many:** If there are two tables, Table A and Table B, a one-to-many (1:N) relationship means a record in Table A can relate to zero, one, or many records in Table B.
- Many records in Table B can relate to one record in Table A.



- *Each customer can place many orders.*
- *Many records in the order table can relate to only one record in the customer table.*

Types of relationships

- **Many-to-many:** If there are two tables, Table A and Table B, a many-to-many (N:N) relationship means many records in Table A can relate to many records in Table B. And many records in Table B can relate to many records in Table A.
- Many-to-many relationships are not kept in a data model. They are broken down into two one-to-many relationships by introducing a junction or middle table.

Customer
customer_id
name
product_id

Product
product_id
description
price
customer_id

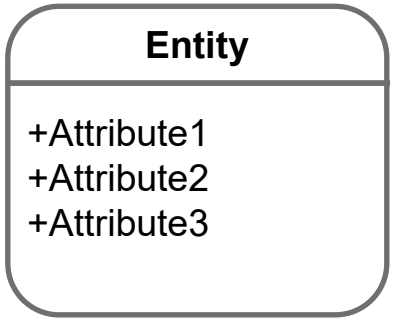
Entity relationship diagrams (ERD)

- The relational database design is very well connected with the entity relationship modelling process including entities, attributes and relationship identification and definition.
- The entity-relationship diagram is commonly used to represent and document the entity relationship models.
- The use of entity relationship diagrams helps to provide the big picture of your database.
- It ensures the data requirements and operations are well defined and documented in your project.

Entity relationship diagrams (ERD)

Entity representation:

- In the ERD, a box with two compartments is used to represent the entity and its related attributes.
- The top compartment represents the entity name, and the bottom compartment includes the related attributes.

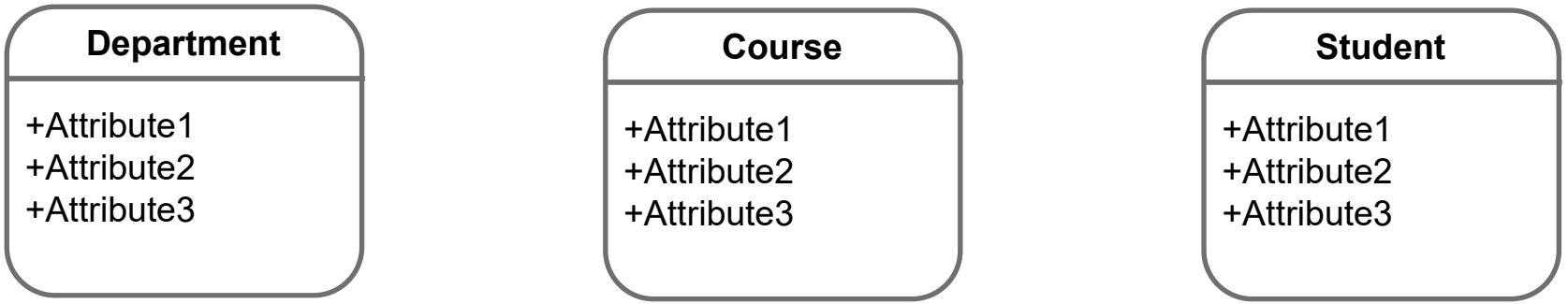


Entity relationship diagrams (ERD)

❖ **Entity representation:**

- Example, a college enrolment system contains a database with information about the students, and the courses available in each department.

➢ You have three entities represented in three separate boxes: the student entity, the course entity, and the department entity.



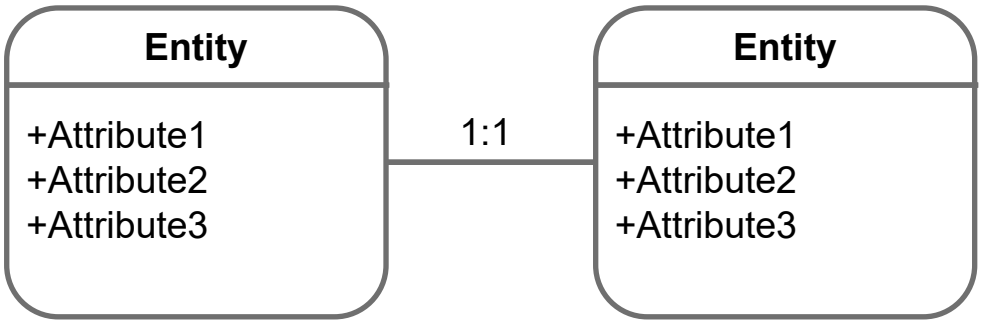
Entity relationship diagrams (ERD)

❖ **Relationship representation**

- The ERD uses different styles of lines to define the distinct types of relationships between entities.
- The line style depends on the cardinality of the relationship, which refers to the number of elements in a set of data as clarified in the following three cases.

❖ **1:1 (one-to-one):** The ER-D uses a straight-line representation for a one-to-one cardinality relationship.

- Example: Each passenger on a train should have only one ticket.

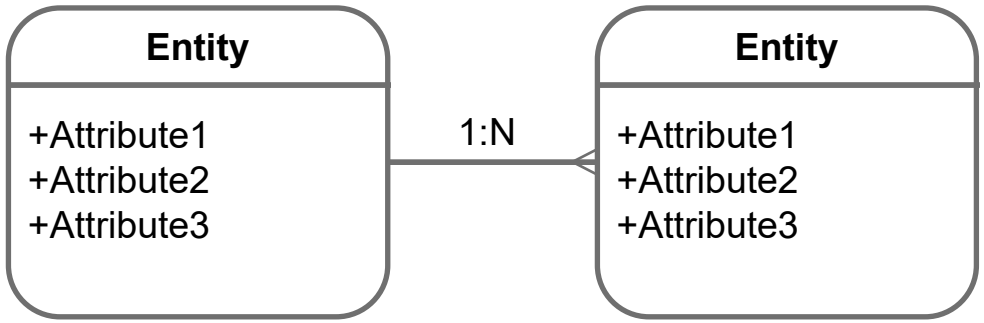


Entity relationship diagrams (ERD)

❖ Relationship representation:

1:N (one-to-many): The ERD is a straight line with a crow's foot notation on one side only to represent a one-to-many cardinality relationship.

- Example: One parent can have many children.

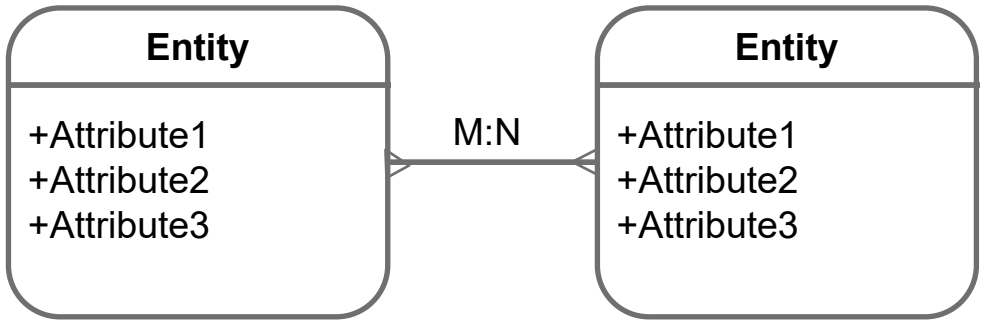


Entity relationship diagrams (ERD)

❖ Relationship representation:

M:N (many-to-many): The ER-D is a straight line with crow's foot notations on both sides of entities to represent a many-to-many cardinality relationship.

Example: Many players play many games.



Entity relationship diagrams (ERD)

❖ **Example:**

How would you depict the relationship between the student, course, and department entities introduced in the college enrolment system example?

Remember that many students may enrol in one course, and one department may offer many courses.

Entity relationship diagrams (ERD)

❖ **Attributes representation:**

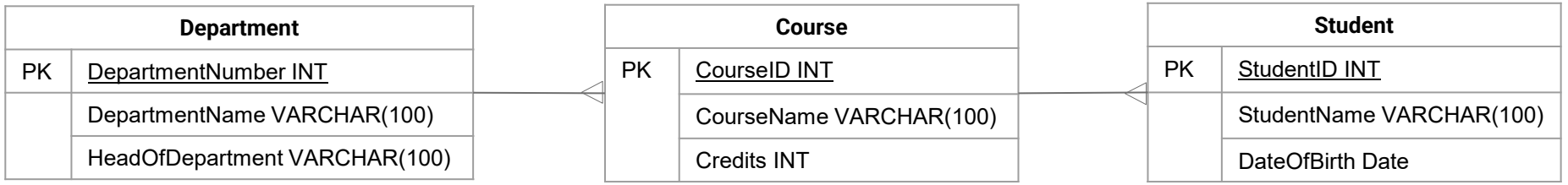
- Each entity has a set of attributes that hold relevant information about it.
- Each attribute must be defined with a data type.

- In the college enrolment example, list the following attributes followed by relevant data types:
  - The department attributes: department number, department name and head of department.
  - The course attributes: course ID, course name, and course credits.
  - The student attributes: student ID, name, and date of birth.

➢ These entities can be listed as three separate tables.

Entity relationship diagrams (ERD)

❖ Attributes representation:



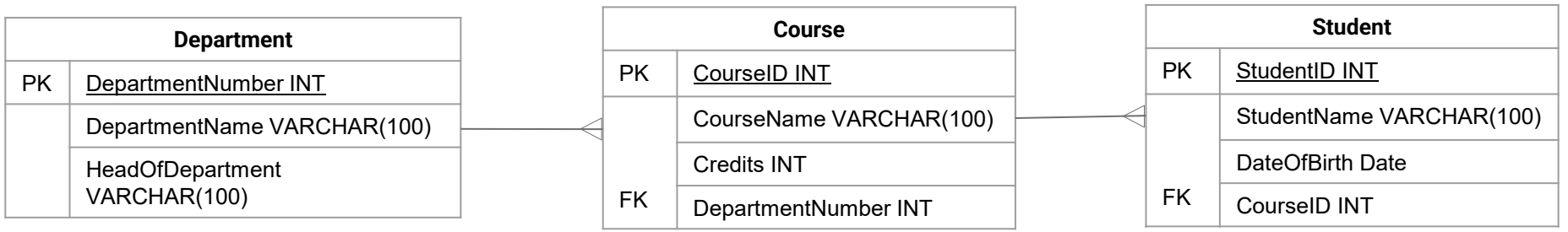
Entity relationship diagrams (ERD)

❖ Attributes representation:

- You need to add the department number to the course table as a **foreign key** in order to link the courses with the departments.
- You need to add the course ID to the student table as a `foreign` **key** in order to link the students with the courses.
- The final college enrolment entity relationship diagram is then three separate tables connected by the relevant keys.

Entity relationship diagrams (ERD)

❖ Attributes representation:



Building a database schema

- Building a database schema is the first step in database design.
- A database schema is like a blueprint of how data in a database will look and be stored.
- A schema consists of what's known as **schema objects** like tables, columns and relationships, at a minimum. Data types, views, stored procedures, primary keys and foreign keys are also schema objects.
- Basically, a database schema consists of:
  - all the important data pertaining to a given scenario and their relationships,
  - unique keys for all entries and database objects,
  - and a name and data type for each column in a table.

Building a database schema

❖ **Scenario:**

❑ To build a database schema for a restaurant booking system, you must consider that:

- Customers make reservations for tables and those tables have orders associated with them.
- An order will have associated menu items that belong to a menu.
- The orders are served by a waiter.

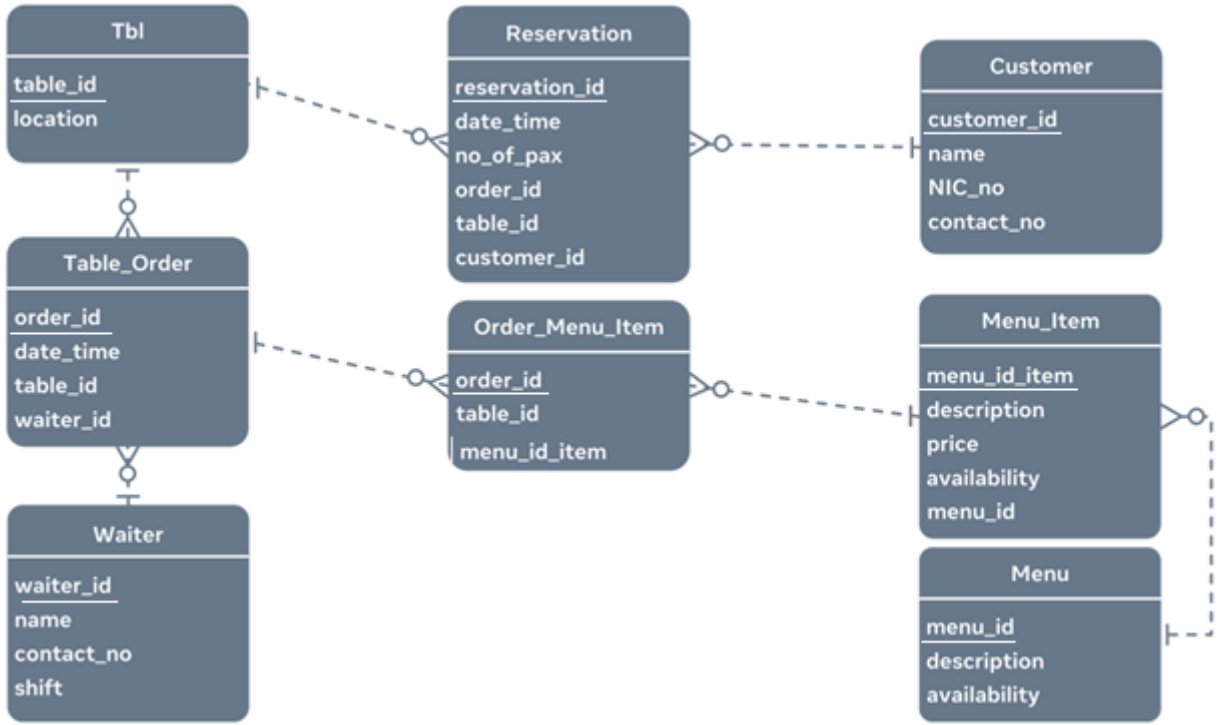
Building a database schema

❖ **The logical database schema:**

❑ Draw an Entity Relationship Diagram (ERD).

Building a database schema

❖ **The logical database schema:**



Building a database schema

❖ The physical database schema

- ❑ Build the physical database schema for the scenario based on the logical database schema.

Building a database schema

❖ **The physical database schema:**

❑ **Step1**: Create the database named "**restaurant** ".

➢ **SQL syntax:** `CREATE DATABASE restaurant;`

❑ Step 2: Create the tables inside "**restaurant** "database.

- The tables and their fields or columns that need to be created are found in the logical database schema.
- Appropriate data types must be used when defining table columns. This enables memory allocation to take place correctly during physical data storage.

Building a database schema

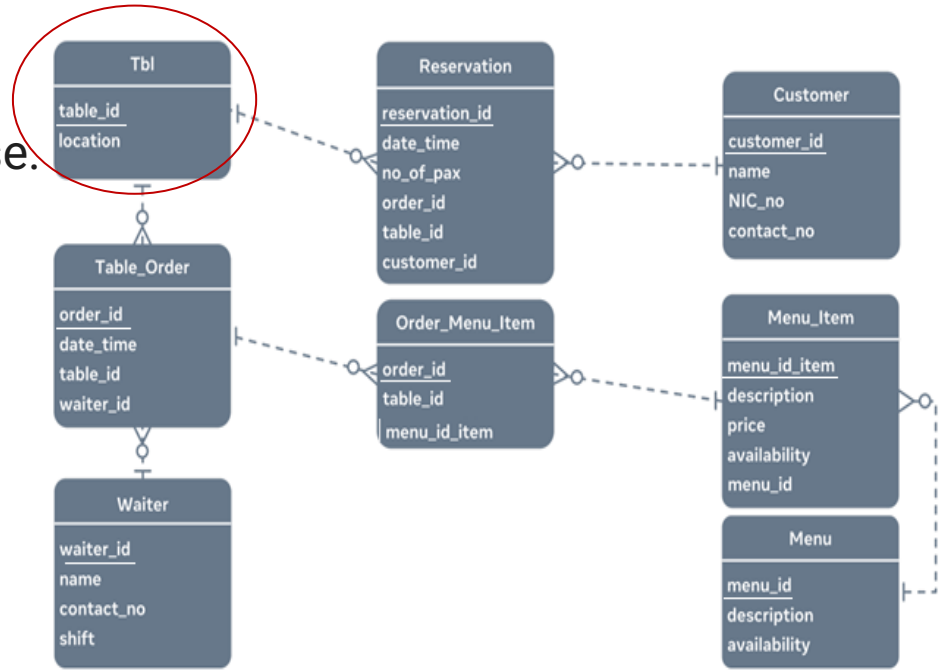
❖ **The physical database schema:**

❑ **Step 2**: Create the tables inside database.

- **'tbl'** table represents a table in restaurant.
- It has a unique ID and a location
- The ID is the primary key of this table.

➢ **SQL syntax:**

```
CREATE TABLE tbl(
table_id INT,
location VARCHAR(255),
PRIMARY KEY (table_id));
```



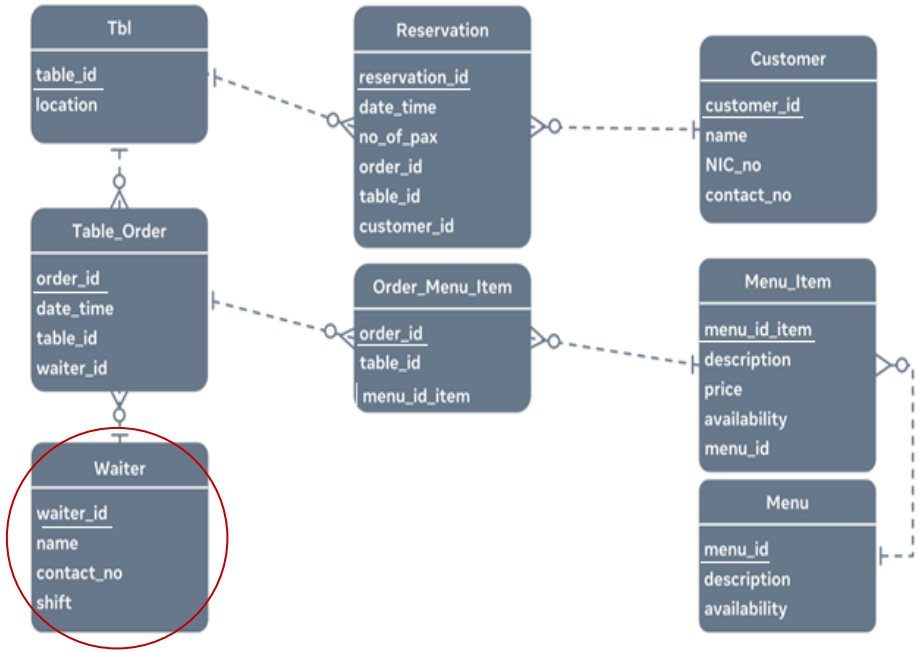
Building a database schema

❖ The physical database schema:

- **"waiter"** table represents the workers' data.
- They have a unique ID, a name, their contact number and which shift they usually work.
- The primary key of the table is the unique ID assigned to the waiter.

➢ **SQL syntax:**

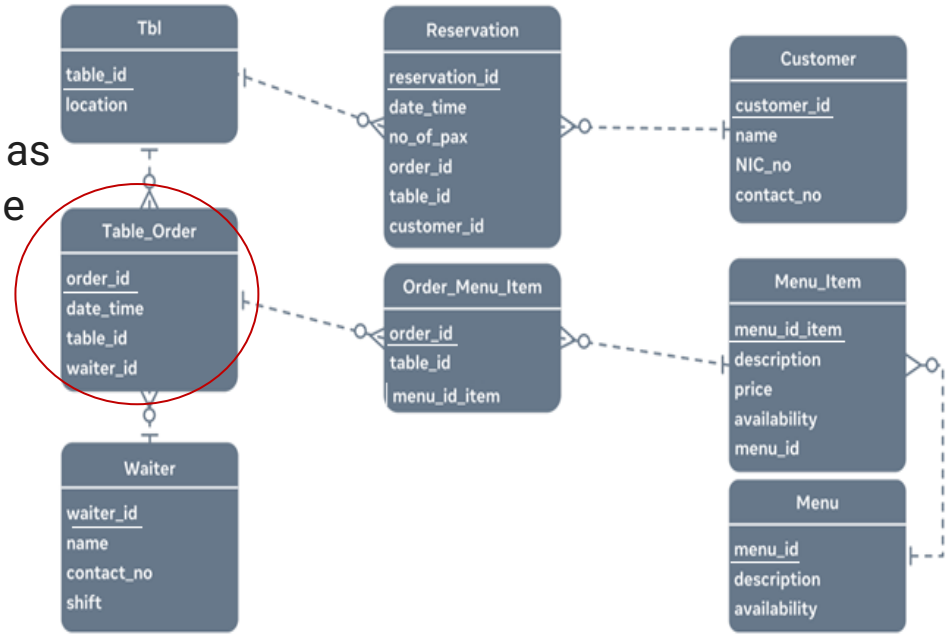
```
CREATE TABLE waiter(
waiter_id INT,
name VARCHAR(150),
contact_no VARCHAR(10),
shift VARCHAR(10),
PRIMARY KEY(waiter_id));
```



Building a database schema

❖ The physical database schema:

- The **"table_order"** table stores data about orders for each table.
- It has the order ID and table ID fields as well as a date_time field to capture the date and time of the order and the ID of the waiter who's supposed to serve that table, for that order.



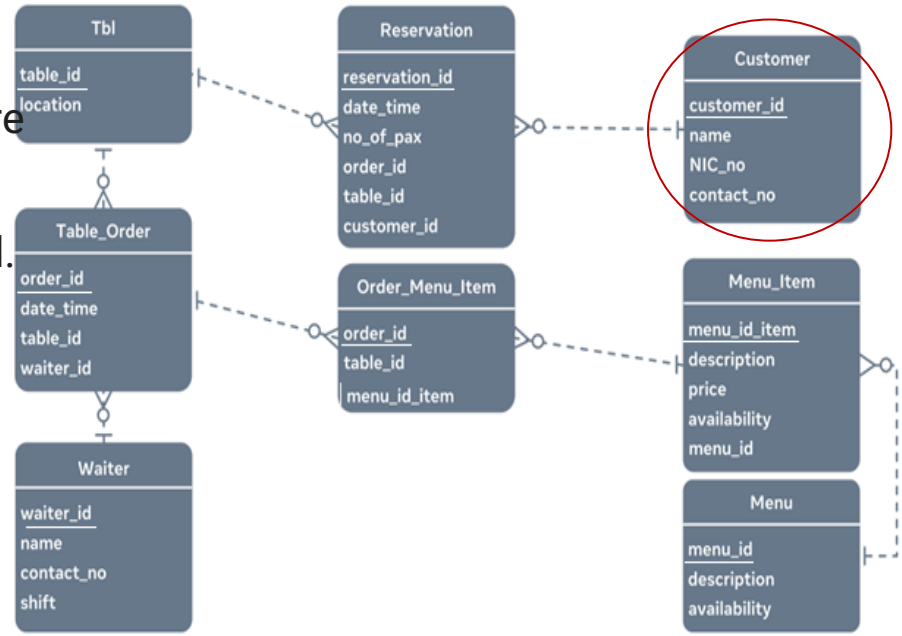
➢ **SQL syntax:**

```
CREATE TABLE table_order(
order_id INT, date_time DATETIME,
table_id INT, waiter_id INT,
PRIMARY KEY (order_id),
FOREIGN KEY (table_id) REFERENCES tbl(table_id),
FOREIGN KEY (waiter_id) REFERENCES waiter(waiter_id));
```

Building a database schema

❖ The physical database schema:

- The "**Customer**" table stores customers' data.
- It has a customer ID, name, NIC number to store the National Identity Card number and contact number fields.
- The primary key is the unique customer ID field.



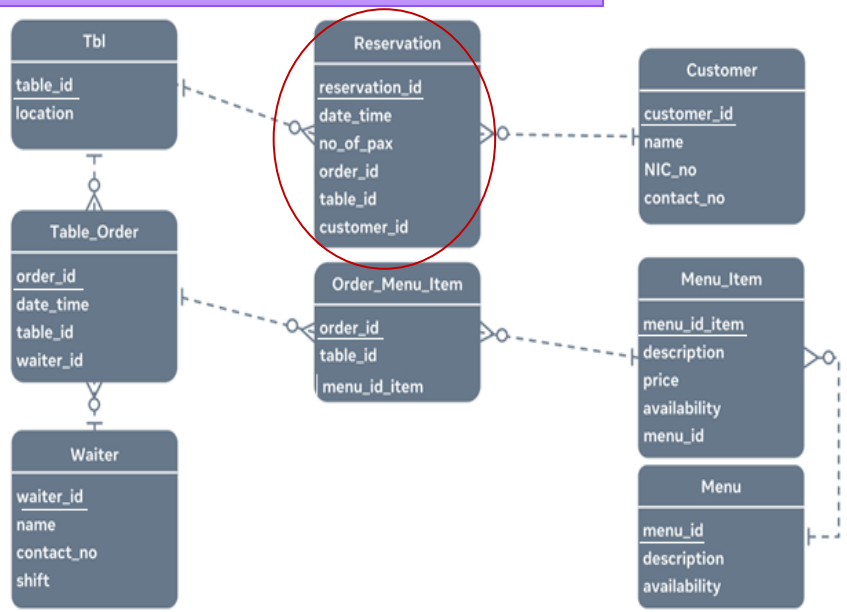
➢ **SQL syntax:**

```
CREATE TABLE customer(
customer_id INT,
name VARCHAR(100),
NIC_no VARCHAR(12),
contact_no VARCHAR(10),
PRIMARY KEY (customer_id));
```

Building a database schema

❖ The physical database schema:

- The "**reservation**" table associates an order with a customer.
- It has a unique ID, a date and time, number of guests or pax expected, the order_id, table_id and the customer_id.
- Its primary key is the unique reservation_id.
- This table is linked with the tbl, table_order and customer tables.



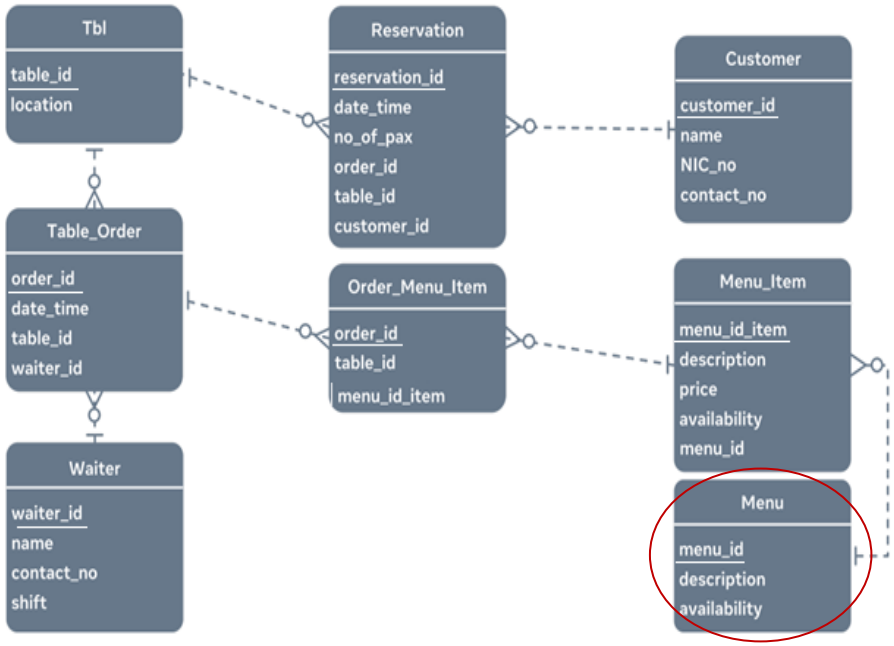
➢ SQL syntax:

```
CREATE TABLE reservation(
reservation_id INT, date_time DATETIME,
no_of_pax INT, order_id INT, table_id INT,
customer_id INT,
PRIMARY KEY (reservation_id),
FOREIGN KEY (order_id) REFERENCES table_order(order_id),
FOREIGN KEY (table_id) REFERENCES tbl(table_id),
FOREIGN KEY (customer_id) REFERENCES customer(customer_id) );
```

Building a database schema

❖ The physical database schema:

- The "**menu**" table stores all the menus of the restaurant.
- It has a menu_id which is the unique field that contains descriptions of the menu and its availability.



➢ SQL syntax:

```
CREATE TABLE menu(
menu_id INT,
description VARCHAR(255),
availability INT,
PRIMARY KEY (menu_id));
```

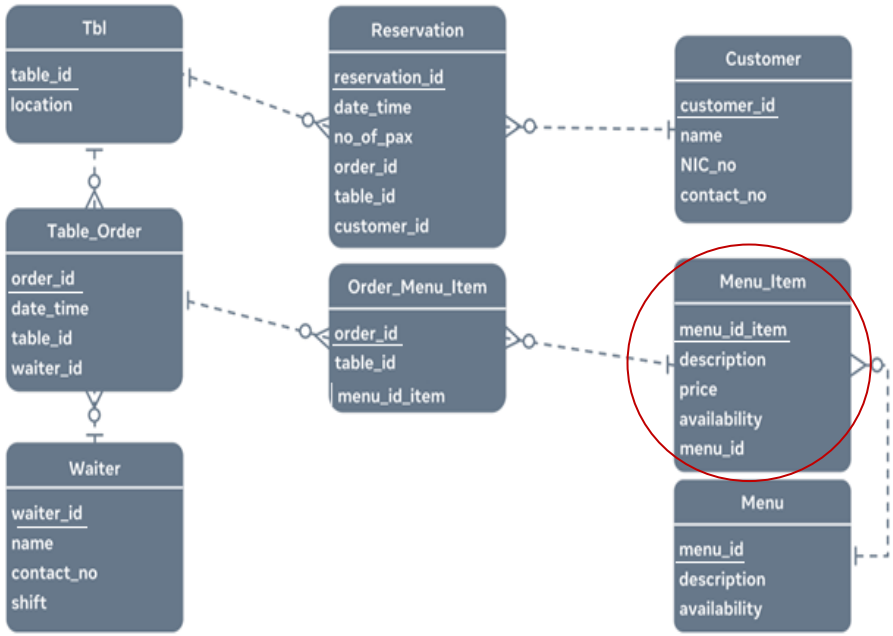
Building a database schema

❖ The physical database schema:

- Every menu can have unique menu items and these menu items are stored against the menu, in the "**menu_item**" table.
- A menu items also has description, price and availability fields.
- This table links with the menu table.

➢ **SQL syntax:**

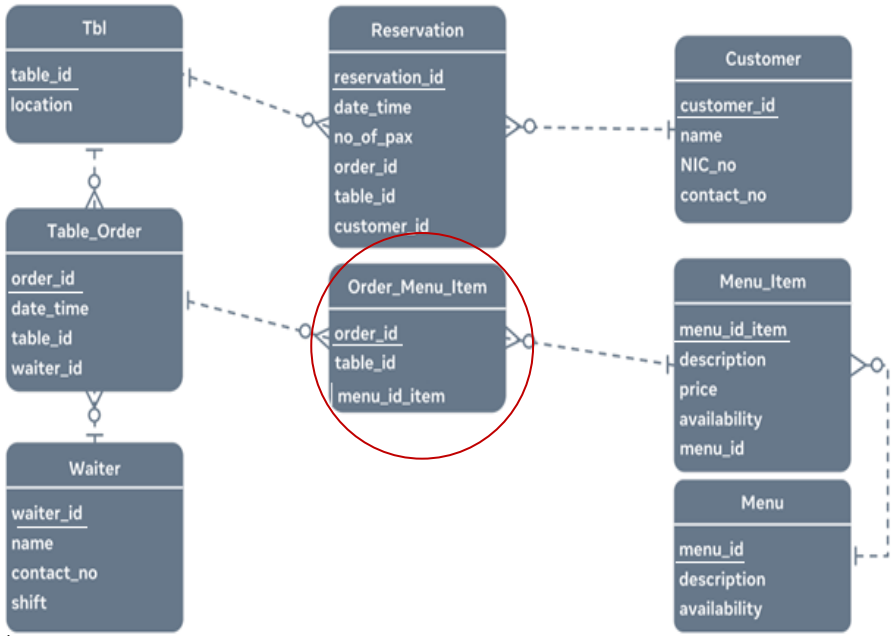
```
CREATE TABLE menu_item(
menu_item_id INT, price FLOAT,
description VARCHAR(255),
availability INT, menu_id INT,
PRIMARY KEY (menu_item_id),
FOREIGN KEY (menu_id) REFERENCES menu(menu_id));
```



Building a database schema

❖ The physical database schema:

- The **"Order_menu_item"** table captures the menu items ordered for a specific order.
- It has the order_id, menu_item_id and the quantity ordered.
- It has a composite primary key of order_id and menu_item_id field combination and its linked with the table_order and menu_item tables.



➢ **SQL syntax:**

```
CREATE TABLE order_menu_item(
order_id INT, menu_item_id INT,
quantity INT,
PRIMARY KEY (order_id, menu_item_id),
FOREIGN KEY (order_id) REFERENCES table_order(order_id
FOREIGN KEY (menu_item_id) REFERENCES menu_item(menu_item_id));
```

Building a database schema: Knowledge check

❖ You're creating a database table for an online store. The table has a column called "Contact details" that stores phone numbers of customers using numeric values only. Any attempts to insert other values result in an error message. What type of constraint is this column an example of?

- ❑ Domain constraint
- ❑ Key constraint
- ❑ Referential integrity constraint

Building a database schema: Knowledge check

❖ You need to ensure that each field in your table's columns contain at least one value. What constraint can you use to enforce this rule?

- ❑ FOREIGN KEY
- ❑ CHECK
- ❑ UNIQUE
- ❑ NOT NULL

Building a database schema: Knowledge check

- You're copying data between tables in your database. Which of the following syntax items must you use to identify the name of the existing table you want to copy?

- ❑ FROM clause
- ❑ SELECT command
- ❑ TABLE keyword